

ASSESSMENT TOOL 1-Pseudocode Writing Task

COURSE NAME: R PROGRAMMING

COURSE CODE: ITA04

QUESTIONS:

1. Variables and Data Types Analysis (BL4 – Analyze)

Design pseudocode to simulate basic variable handling in R programming:

- Declare variables of different data types (numeric, character, logical)
- Perform arithmetic and logical operations
- Display the results

CODE:

```
num_var <- 25.5
char_var <- "R Programming"
logical_var <- TRUE
```

```
print(num_var)
print(char_var)
print(logical_var)
```

```
class(num_var)
class(char_var)
class(logical_var)
```

```
sum_result <- num_var + 10
product_result <- num_var * 2
print(sum_result)
print(product_result)
```

```
and_result <- TRUE & FALSE
or_result <- TRUE | FALSE
not_result <- !logical_var
```

```
print(and_result)
print(or_result)
print(not_result)
```

OUTPUT:

```
> num_var <- 25.5
> char_var <- "R Programming"
> logical_var <- TRUE
>
> print(num_var)
[1] 25.5
> print(char_var)
[1] "R Programming"
> print(logical_var)
[1] TRUE
>
> class(num_var)
[1] "numeric"
> class(char_var)
[1] "character"
> class(logical_var)
[1] "logical"
>
> sum_result <- num_var + 10
> product_result <- num_var * 2
> print(sum_result)
[1] 35.5
> print(product_result)
[1] 51
>
> and_result <- TRUE & FALSE
> or_result <- TRUE | FALSE
> not_result <- !logical_var
>
> print(and_result)
[1] FALSE
> print(or_result)
[1] TRUE
> print(not_result)
[1] FALSE
> |
```

2. Data Structures Implementation (BL4 – Analyze)

Write pseudocode to demonstrate advanced data structures in R:

- Create and initialize a vector, list, and matrix
- Perform operations such as indexing, updating, and aggregation
- Compare the structures based on usage

CODE:

```
my_vector <- c(10, 20, 30, 40, 50)
print(my_vector)
my_vector[2] <- 99
print(my_vector)
sum(my_vector)
mean(my_vector)
```

```
my_list <- list(name = "John", age = 25, marks = c(80, 90, 85))
print(my_list)
my_list$age <- 26
print(my_list$age)
```

```
my_matrix <- matrix(1:6, nrow = 2, ncol = 3)
print(my_matrix)
my_matrix[1, 2]
my_matrix[1, 2] <- 100
print(my_matrix)
```

```
rowSums(my_matrix)
colSums(my_matrix)
```

OUTPUT:

```
> my_vector <- c(10, 20, 30, 40, 50)
> print(my_vector)
[1] 10 20 30 40 50
> my_vector[2] <- 99
> print(my_vector)
[1] 10 99 30 40 50
> sum(my_vector)
[1] 229
> mean(my_vector)
[1] 45.8
>
> my_list <- list(name = "John", age = 25, marks = c(80, 90, 85))
> print(my_list)
$name
[1] "John"

$age
[1] 25

$marks
[1] 80 90 85

> my_list$age <- 26
> print(my_list$age)
[1] 26
>
> my_matrix <- matrix(1:6, nrow = 2, ncol = 3)
> print(my_matrix)
     [,1] [,2] [,3]
[1,]    1    3    5
[2,]    2    4    6
> my_matrix[1, 2]
[1] 3
> my_matrix[1, 2] <- 100
> print(my_matrix)
     [,1] [,2] [,3]
[1,]    1  100    5
[2,]    2    4    6
> rowSums(my_matrix)
[1] 106  12
> colSums(my_matrix)
[1]    3  104  11
> |
```

3. Matrix and Array Operations (BL4 – Analyze)

Design pseudocode to perform operations on matrices and arrays:

- Create a matrix and a 3D array
- Perform addition, multiplication (matrix), and element-wise operations
- Display the results and analyze differences

CODE:

```
matrix_a <- matrix(1:4, nrow = 2, ncol = 2)
matrix_b <- matrix(5:8, nrow = 2, ncol = 2)
```

```
print(matrix_a)
print(matrix_b)
```

```
matrix_sum <- matrix_a + matrix_b
print(matrix_sum)
```

```
matrix_product <- matrix_a %*% matrix_b
print(matrix_product)
```

```
elementwise_product <- matrix_a * matrix_b
print(elementwise_product)
```

```
my_array <- array(1:24, dim = c(2, 3, 4))
print(my_array)
dim(my_array)
my_array[1, 2, 3]
```

OUTPUT:

```
[1,] [,1] [,2]
[2,]    6  10
      8  12
>
> matrix_product <- matrix_a %*% matrix_b
> print(matrix_product)
      [,1] [,2]
[1,]   23   31
[2,]   34   46
>
> elementwise_product <- matrix_a * matrix_b
> print(elementwise_product)
      [,1] [,2]
[1,]    5   21
[2,]   12   32
>
> my_array <- array(1:24, dim = c(2, 3, 4))
> print(my_array)
, , 1
      [,1] [,2] [,3]
[1,]    1    3    5
[2,]    2    4    6
, , 2
      [,1] [,2] [,3]
[1,]    7    9   11
[2,]    8   10   12
, , 3
      [,1] [,2] [,3]
[1,]   13   15   17
[2,]   14   16   18
, , 4
      [,1] [,2] [,3]
[1,]   19   21   23
[2,]   20   22   24
> dim(my_array)
[1] 2 3 4
> my_array[1, 2, 3]
[1] 15
> |
```

4. Data Frame Construction and Manipulation (BL4 – Analyze))

A dataset needs to be organized efficiently.

Write pseudocode to:

- Create a data frame with multiple columns (numeric and character)
- Add, delete, and modify columns
- Perform basic summary operations

CODE:

```
student_df <- data.frame(  
  ID = c(1, 2, 3),  
  Name = c("Alice", "Bob", "Charlie"),  
  Marks = c(85, 90, 78)  
)
```

```
print(student_df)
```

```
student_df$Grade <- c("A", "A", "B")  
print(student_df)
```

```
student_df$Marks <- NULL  
print(student_df)
```

```
student_df$Marks <- c(85, 90, 78)  
student_df$Marks[2] <- 95  
print(student_df)
```

```
summary(student_df$Marks)  
mean(student_df$Marks)  
nrow(student_df)  
ncol(student_df)  
str(student_df)
```

OUTPUT:

```
+ Marks = c(85, 90, 78)
+ )
>
> print(student_df)
  ID   Name Marks
1  1   Alice    85
2  2     Bob    90
3  3  Charlie    78
>
> student_df$Grade <- c("A", "A", "B")
> print(student_df)
  ID   Name Marks Grade
1  1   Alice    85     A
2  2     Bob    90     A
3  3  Charlie    78     B
>
> student_df$Marks <- NULL
> print(student_df)
  ID   Name Grade
1  1   Alice     A
2  2     Bob     A
3  3  Charlie     B
>
> student_df$Marks <- c(85, 90, 78)
> student_df$Marks[2] <- 95
> print(student_df)
  ID   Name Grade Marks
1  1   Alice     A    85
2  2     Bob     A    95
3  3  Charlie     B    78
>
> summary(student_df$Marks)
  Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
  78.0   81.5   85.0   86.0   90.0   95.0
> mean(student_df$Marks)
[1] 86
> nrow(student_df)
[1] 3
> ncol(student_df)
[1] 4
> str(student_df)
'data.frame':   3 obs. of  4 variables:
 $ ID   : num  1 2 3
 $ Name : chr  "Alice" "Bob" "Charlie"
 $ Grade: chr  "A" "A" "B"
 $ Marks: num  85 95 78
> |
```

5. Functions and Logical Operations (BL4 – Analyze)

Design pseudocode to implement functions and logical vector operations:

- Create a user-defined function with input parameters
- Perform operations on logical vectors
- Return and display computed results

CODE:

```
calculate_stats <- function(x, y) {
  total <- x + y
  difference <- x - y
  product <- x * y
  result <- list(sum = total, diff = difference, prod = product)
  return(result)
}
```

```
output <- calculate_stats(10, 5)
print(output)
```

```
logical_vec <- c(TRUE, FALSE, TRUE, TRUE, FALSE)
```

```
print(logical_vec)
sum(logical_vec)
```

```
any(logical_vec)
all(logical_vec)
```

```
check_even <- function(num) {
  if (num %% 2 == 0) {
    return(TRUE)
  } else {
    return(FALSE)
  }
}
```

```
result1 <- check_even(10)
result2 <- check_even(7)
print(result1)
print(result2)
```

```
numbers <- c(1, 2, 3, 4, 5, 6, 7, 8)
even_check <- numbers %% 2 == 0
print(even_check)
numbers[even_check]
```

OUTPUT:

```
+ }
>
> output <- calculate_stats(10, 5)
> print(output)
$sum
[1] 15

$diff
[1] 5

$prod
[1] 50

>
> logical_vec <- c(TRUE, FALSE, TRUE, TRUE, FALSE)
>
> print(logical_vec)
[1] TRUE FALSE TRUE TRUE FALSE
> sum(logical_vec)
[1] 3
> any(logical_vec)
[1] TRUE
> all(logical_vec)
[1] FALSE
>
> check_even <- function(num) {
+   if (num %% 2 == 0) {
+     return(TRUE)
+   } else {
+     return(FALSE)
+   }
+ }
>
> result1 <- check_even(10)
> result2 <- check_even(7)
> print(result1)
[1] TRUE
> print(result2)
[1] FALSE
>
> numbers <- c(1, 2, 3, 4, 5, 6, 7, 8)
> even_check <- numbers %% 2 == 0
> print(even_check)
[1] FALSE TRUE FALSE TRUE FALSE TRUE FALSE TRUE
> numbers[even_check]
[1] 2 4 6 8
> |
```